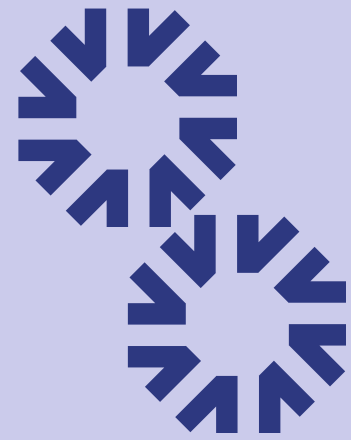


Transfer Learning

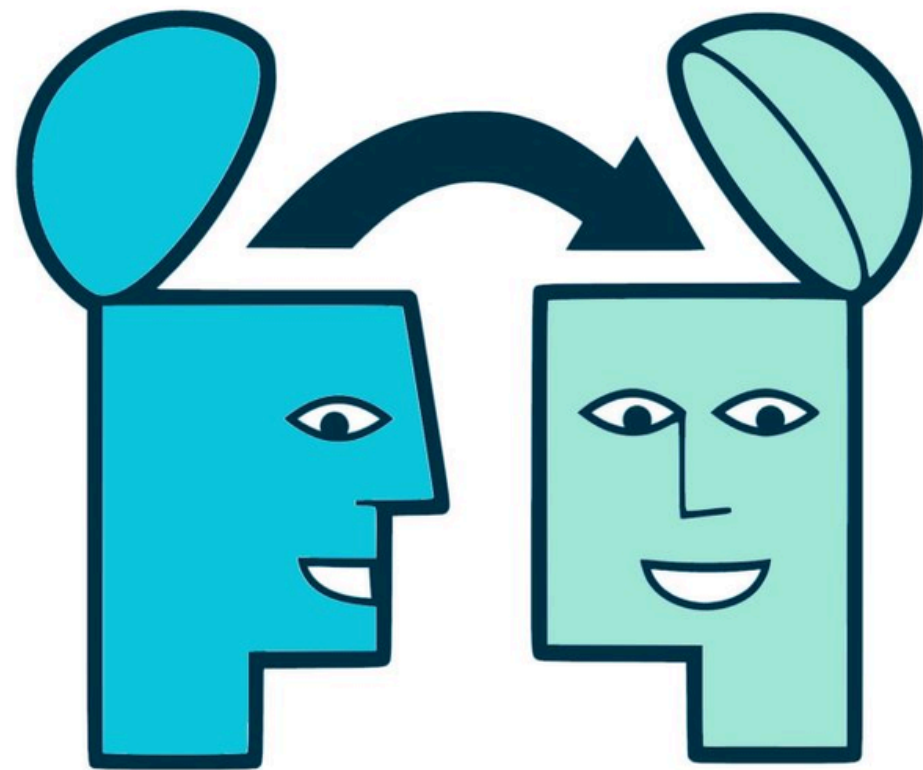
By:
Eraj Zehra
Hira Tanveer

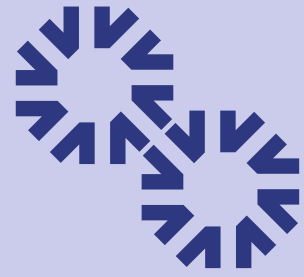




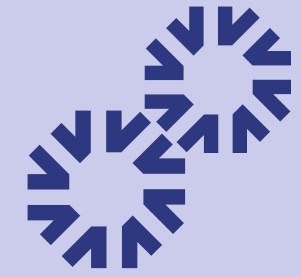
What is transfer learning

- Transfer learning is the reuse of a pre-trained model on a new but related problem.
- Instead of training a model from scratch, we start with a model that has already learned useful patterns from a large dataset.
- The idea is to transfer the learned knowledge (features, patterns, relationships) from one task (Task A) to another task (Task B).
- A pre-trained model already understands low-level patterns like edges, shapes, textures, object boundaries, etc.
- These learned features help the model perform well on a new problem even with limited data.
- Example: A model trained to detect food in images already learns edges, textures, shapes, and colours. These general features help it recognize drinks or utensils, even without starting over.





When to Use?



- Limited Training Data

Transfer learning works best when you don't have enough labelled data to train a deep model from scratch.

Example: 1,000 Urdu tweets, or 2,000 product photos – small datasets but enough for fine-tuning.

- A Pretrained Model Exists for a Similar Task

If a high-quality pretrained model already exists, reuse it instead of reinventing the wheel.

Examples: ImageNet models for medical images, BERT for sentiment.

- Both Tasks Have the Same Input Type

Transfer learning requires similar input formats (image→image, text→text).

Examples: Image classification → segmentation; English language modelling → Urdu sentiment analysis.

- Training From Scratch Is Too Costly

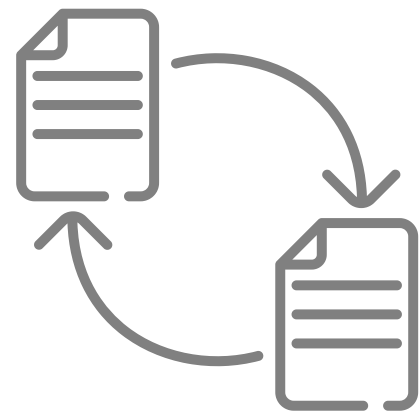
Use transfer learning when training is too slow, expensive, or requires huge compute resources. It saves time, GPU cost, and energy.

- The Learned Features are general enough to Transfer

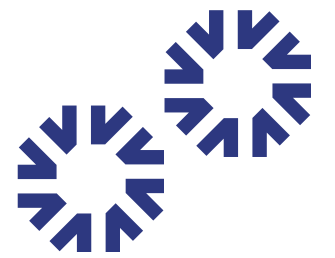
Works only when the source task learns general patterns that can help the new task.

Successful: ImageNet → medical imaging, BERT → sentiment analysis

Not successful: Cat classification → speech recognition (completely unrelated domains)



Background of Transfer Learning



- Traditional ML vs. Transfer Learning

Traditional machine learning trains models from scratch for every task.

Needs large labelled datasets

High computational cost and training time

- Transfer learning reuses pretrained models, reducing the need for massive datasets and accelerating training.

- Early Developments & Motivation

Initial studies on “Learning to Learn” in 1995 (NIPS workshop) focused on lifelong learning and knowledge reuse.

The goal was to build models that retain and apply knowledge from previous tasks to new ones.

Motivated by real-world problems where labelled data is limited, and tasks vary in domain or distribution.

- Relation to Other Learning Paradigms

Connected concepts include: multi-task learning, meta-learning, knowledge transfer, and incremental/lifelong learning.

- Multi-task learning: learns multiple tasks simultaneously through shared features.

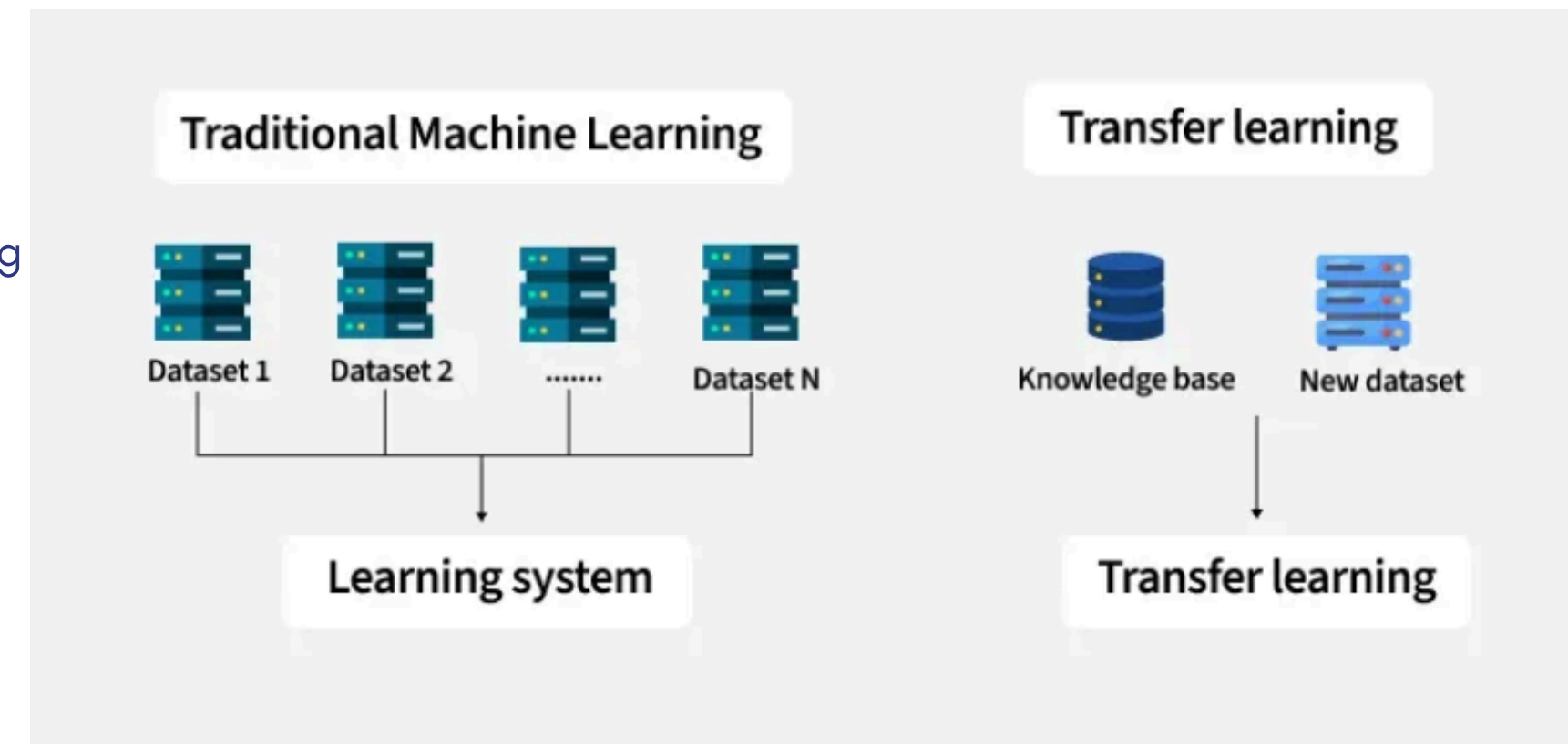
Transfer learning: focuses on a target task, transferring knowledge from source tasks.

- Rise with Deep Learning

Deep neural networks have millions of parameters and need huge datasets to train effectively.

Pretrained models on large datasets like Wikipedia became common.

Transfer learning enables these models to be adapted for new tasks with limited data.



Definition of a Domain

Domain describes what kind of data is there and how that data is distributed.

$$D = \{\mathcal{X}, P(X)\}$$

where:

- $\mathcal{X}$ is the feature space
- $P(X)$ is the marginal probability distribution over the features



Source Domain:

This is the domain from which existing knowledge is transferred. It usually has a large amount of labelled data, and a model is pre-trained on it to learn general features or parameters.



Target Domain:

This is the new, related domain where you want to apply the knowledge learned from the source domain. The key characteristic of the target domain is that it typically has insufficient labeled data for training a robust model from scratch.

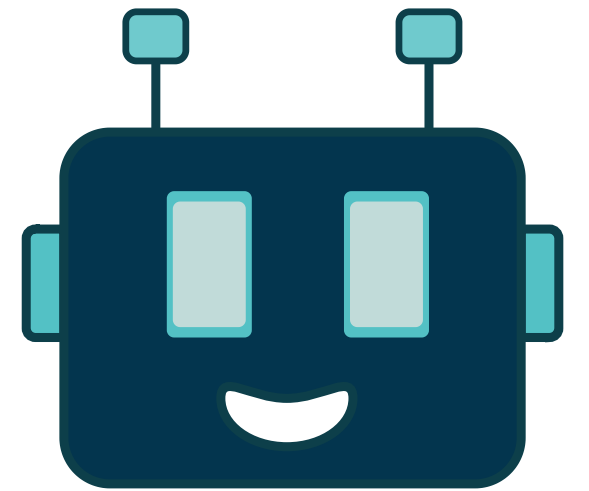
Definition of a Task

A task defines what we want the model to do. It has two components.

Label space Y – the possible outputs the model can predict.

Predictive function $f(\cdot)$ – the function that maps an input x to a label y .

The function is unknown initially; the model learns it from training data.



How the predictive function works?

Once trained, the function $f(x)$ predicts the probability of the label of a an input.

$$f(x) = P(y \mid x)$$

Domain Notation



Source Domain (DS)

The source data is written as:

$$D_S = \{(x_1^S, y_1^S), \dots, (x_{n_S}^S, y_{n_S}^S)\}$$

This means:

- There are n_S labeled examples in the source dataset.
- Each example has:
 - $x_i^S \in X_S$: the input features (images, text, etc.)
 - $y_i^S \in Y_S$: the labels (classes)

Target Domain (DT)

The target data is written as:

$$D_T = \{(x_1^T, y_1^T), \dots, (x_{n_T}^T, y_{n_T}^T)\}$$

This means:

- There are n_T labeled examples in the target dataset.
- Each example has:
 - $x_i^T \in X_T$: new inputs (could be different from source)
 - $y_i^T \in Y_T$: new labels

The target dataset has very few labeled examples. Often much fewer than the source dataset

$$0 \leq n_T \ll n_S$$

When Transfer learning applies?

1. The domains are different:

$$D_S \neq D_T$$

- Source: A model trained on millions of general images (like ImageNet) to classify everyday objects.
- Target: A small dataset of X-ray images for classifying diseases.

2. The tasks are different:

$$T_S \neq T_T$$

- Source: A model trained on millions of general images (like ImageNet) to classify everyday objects (e.g., cycle, shoes, bottle).
- Target: A dataset of similar natural images, but the new task is to detect objects in the image (e.g., locating shoes or carrot with bounding boxes).

There are two types of tasks:

Source task (TS): The prediction task you originally learned.

Target task (TT): The new prediction task you want to learn.



Types of Transfer learning

There are three types of transfer learning:

Inductive Transfer

Source and target tasks differ, domains may be same or different.

Using an ImageNet-pretrained model (task: object classification) and fine-tuning it for object detection on a smaller dataset.

Unsupervised Transfer

Source and target tasks differ, but no labeled data is available.

Learning normal behavior patterns from unlabeled transaction data and then detecting unusual patterns as fraud.

Transductive Transfer

Source and target tasks are the same, but domains (datasets) differ. Source data is labeled, target data usually unlabeled.

Training a sentiment classifier on restaurant reviews and applying it to classify movie reviews.

How Transfer learning works

- Start with a pretrained model that already learned general visual features from a large dataset.
- Remove the model's original final classification layer because its labels don't match the new task.
- Add a new classification layer with randomly initialized weights and biases for the target task.
- Freeze the pretrained convolution layers so their weights stay unchanged.
- Train only the new classification layer to learn the target task's labels.
- Optionally unfreeze and fine-tune deeper layers to adapt features to the new domain.
- Use the adapted model to map input features to the new target labels.

Example

1. Source Model (Pretrained on Natural Images)

Source Domain (DS):

Millions of natural images (ImageNet):



- Input examples:

- x_1^S : photo of a dog
- x_2^S : photo of a car
- x_3^S : photo of a tree

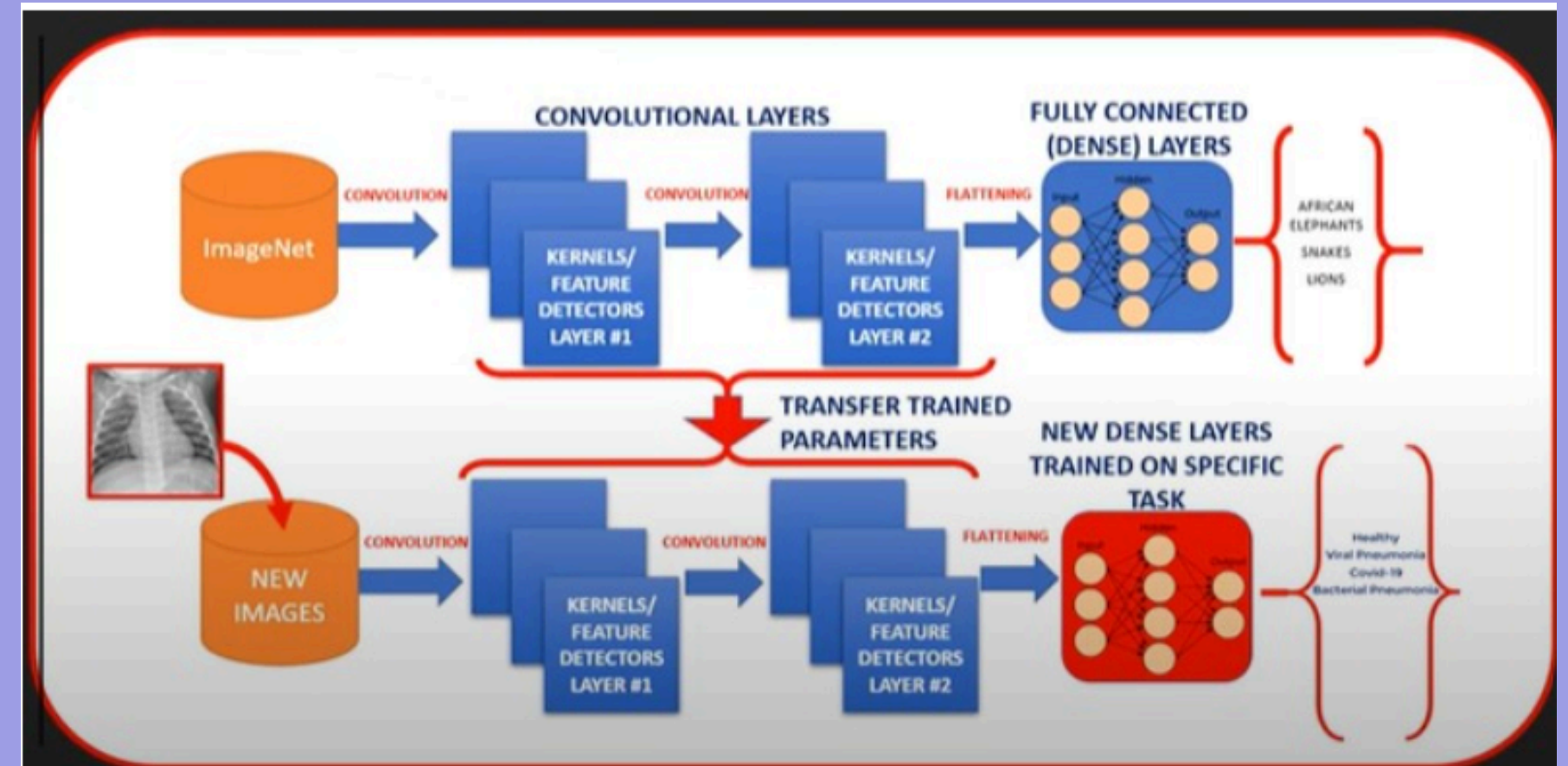
Source Task (TS):

Object classification – predicting what object is in a natural image.

Output examples (labels):

- $y_1^S = \text{"dog"}$
- $y_2^S = \text{"car"}$
- $y_3^S = \text{"tree"}$

Layer Type	Learns
Early layers	Edges, corners, gradients
Middle layers	Shapes, textures, contours
Late layers	Very specific natural-image features (eyes, fur, wheels, faces)



Example Cotd.

2. Target Model (Fine-Tuned on X-ray Images)

Target Domain (DT):

Small dataset of labeled chest X-ray images:

- Input examples:
 - x_1^T : lung X-ray showing pneumonia
 - x_2^T : lung X-ray showing no disease
 - x_3^T : X-ray showing COVID-19 pattern



Target Task (TT):

Medical image classification – predicting disease type from an X-ray.

Use the pre-trained model

- Start with a CNN already trained on a large dataset like ImageNet.
- This model has learned strong general features (edges, textures, shapes).

Remove the last layer (fully connected layer)

- The original final layer outputs **1000 ImageNet classes**, which are not useful for medical tasks.
- So we delete/remove this layer.

Add a new classification layer

- Add a new **Fully Connected (FC) layer** that matches the medical labels (e.g., 3 classes).
- This new layer is the only part of the model that starts fresh.

Initialize weights and biases randomly

- The new classification layer has:
 - **New weights (randomly initialized)**
 - **New biases (randomly initialized)**
- It has no prior knowledge, so train it fully.

Freeze the convolution layers

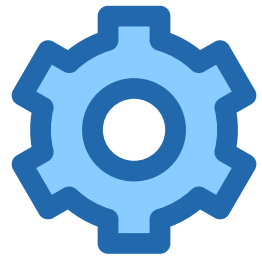
- All convolution layers (feature extractor) keep their pre-trained ImageNet weights.
- These layers remain frozen = weights do not update during training.

Train ONLY the new classification layer

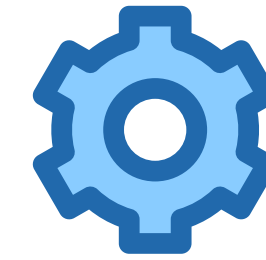
- During training, gradients only update the new FC layer.
- The convolutional base stays unchanged.

Target Model Output:

- "pneumonia"
- "normal"
- "COVID-19"



Frozen vs Trainable Layers



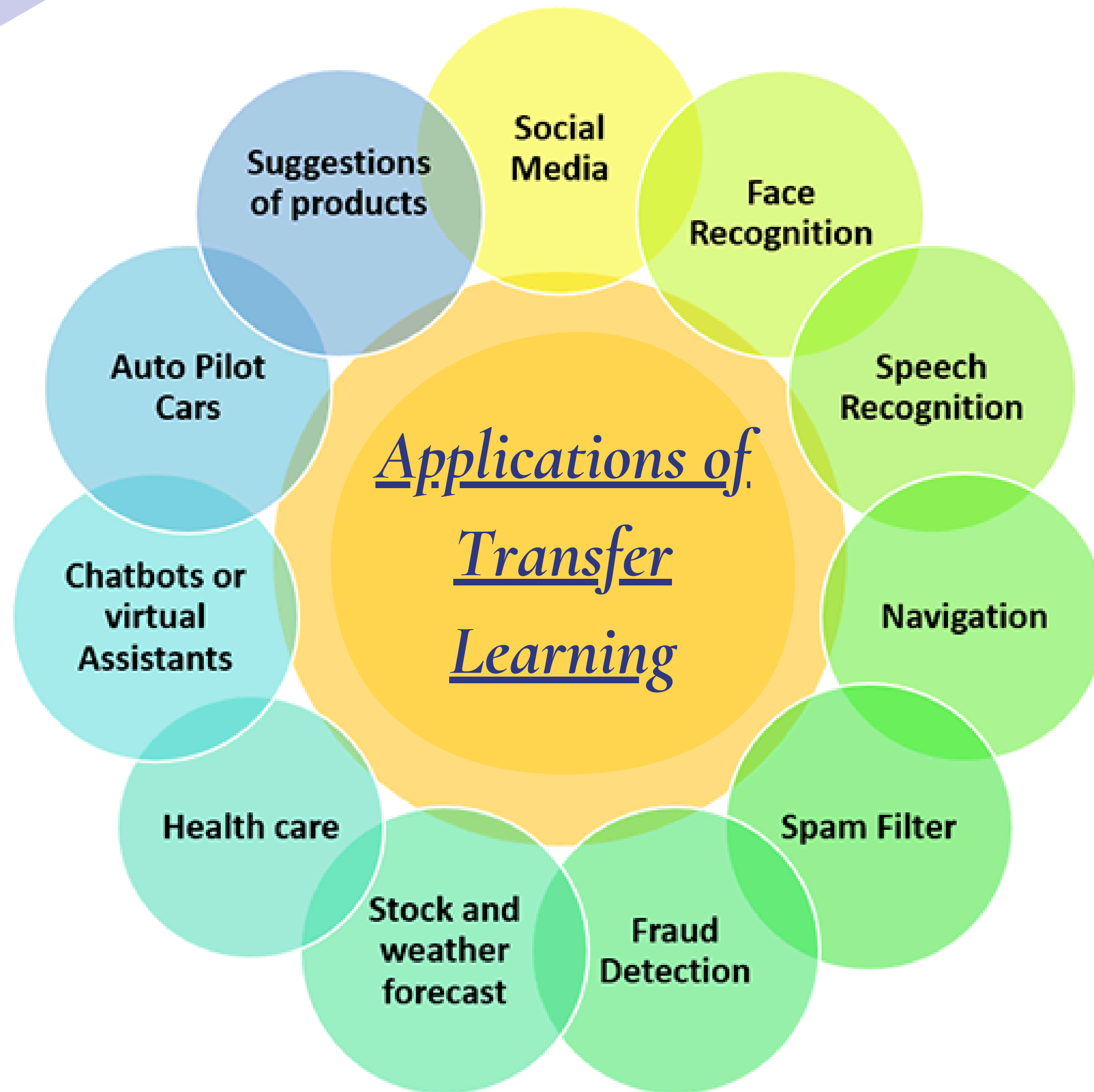
Aspect	Frozen Layers	Trainable Layers
Definition	Layers whose weights are kept fixed and not updated during training	Layers whose weights are updated during training
Purpose	Preserve general features learned from large pre-trained datasets	Adapt to task-specific features of the new dataset
Learning Process	No backpropagation updates; remain constant	Updated through backpropagation based on new data
Use Case	Used when new dataset is small or similar to the original dataset	Used when new dataset is large or significantly different from the original task
Computation Cost	Lower, since fewer parameters are trained	Higher, as more parameters need to be updated
Example in CNN	Early convolutional layers that capture edges, textures and basic shapes	Later fully connected layers or deeper convolutional layers for fine-tuned features

Advantages

- 1.Reduced Training Time: Uses pre-learned knowledge, so models train much faster.
- 2.Lower Data Requirements: Performs well even with small labelled datasets.
- 3.Improved Accuracy: Often boosts performance compared to training from scratch.
- 4.Better Generalisation: Captures richer features learned from large source datasets.
- 5.Cost-Effective: Saves computational and resource costs.
- 6.Helps in Complex Tasks: Works well for tasks where collecting huge data is difficult.
- 7.Faster Convergence: Model reaches optimal performance in fewer epochs.
- 8.Prevents Overfitting: Reduces overfitting when target data is limited.

Disadvantages

- 1.Domain Mismatch Risk: Poor performance if source and target domains differ too much.
- 2.Negative Transfer: Transfer can hurt accuracy instead of helping when tasks are unrelated.
- 3.Dependence on Pretrained Models: Requires access to good pretrained weights.
- 4.Computational Load for Fine-Tuning: Large pretrained models can be heavy to adjust.
- 5.Limited Flexibility: Pretrained features may not fully fit specific target problems.
- 6.Bias Transfer: Any bias in the source dataset is carried into the target model.
- 7.Model Complexity: Managing layers to freeze/unfreeze can be tricky.
- 8.Storage Requirements: Pretrained models may require large storage space.



References:

1. <https://builtin.com/data-science/transfer-learning>
2. <https://www.ibm.com/think/topics/transfer-learning>
3. [https://www.cse.ust.hk/~qyang/Docs/2009/tkde transfer learning .pdf](https://www.cse.ust.hk/~qyang/Docs/2009/tkde_transfer_learning.pdf)
4. <https://www.geeksforgeeks.org/machine-learning/ml-introduction-to-transfer-learning/>
5. <https://ieeexplore.ieee.org/document/10025748>



Thank You!